

From Entropy to Epiplexity

Rethinking Information for Computationally Bounded Intelligence

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson

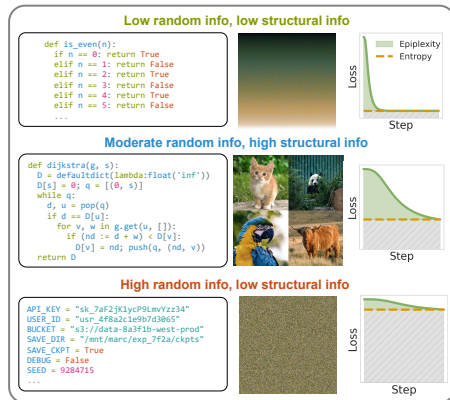
Carnegie Mellon University, New York University

arXiv:2601.03220, version 2, 16 March 2026

Speaker: Alex Kravatsky

Which pretraining data support transfer?

- Models pretrained on text can improve performance on robotics, theorem proving and forecasting tasks.
- Dataset size alone does not measure reusable predictive structure.
- **Question:** how much structure can a particular learner extract, given a fixed compute budget?



Training loss on three kinds of data. Dashed line: final loss; green area: loss removed by learning. Source: Fig. 1 of Finzi et al. (2026).

Two classical lenses on information

$H(X) = \mathbb{E}[-\log_2 p(X)]$ is the Shannon entropy of a random variable X with probability mass function p . It is the ideal average code length in bits for a draw from the distribution.

$K(x)$ is the Kolmogorov complexity of one realised string x : the length of the shortest self-delimiting program that outputs x on a fixed universal computer $\mathcal{U}$ (Li and Vitányi, 2008).

Shannon entropy

Uncertainty in a distribution. Uniform n -bit strings have entropy n ; a distribution concentrated on one string has entropy zero.

Kolmogorov complexity

Program length for one string. A repeating pattern can be specified by its rule and length; a typical random string needs about n bits.

Kolmogorov complexity is a mathematical benchmark; no algorithm computes $K(x)$ exactly for every string.

Entropy measures average uncertainty; Kolmogorov complexity measures program length.

The missing ingredient: computational access

Neither classical quantity asks whether a learner can find or use the relevant pattern within a finite budget. For example, a deterministic training run can produce a network with useful circuits, although the rules and random seed form a short description of the run.

- A weak observer may treat a structured process as random.
- A stronger observer may discover a compact predictive model.
- We therefore define information relative to the models available within time T .

Epiplexity measures model bits in the best description available under a time bound.

A model with bounded computation

Fix a prefix-free universal computer $\mathcal{U}$ and a time bound $T(n)$. The observer may use only programs $P \in \mathcal{P}_T$ that, within $T(n)$ steps, can both

- evaluate a probability $P(x)$ for an observation x ;
- sample a new x from the same distribution P .

The program length $|P|$ is measured in bits. This is the idealised definition; in experiments we replace $\mathcal{P}_T$ by models reachable with a stated architecture, optimiser and FLOP budget.

The formal bound limits evaluation and sampling; empirical estimates also account for training.

Epiplexity: the two-part description

Definition 1 (Def. 8 of the paper)

For a random variable X on $\{0, 1\}^n$, let

$$P^* = \arg \min_{P \in \mathcal{P}_T} \{|P| + \mathbb{E}[-\log P(X)]\},$$

with ties broken by the shortest program. Logarithms are base 2. The *epiplexity* of X is $S_T(X) := |P^*|$. The *time-bounded entropy* of X is $H_T(X) := \mathbb{E}[-\log P^*(X)]$.

S_T is the model part: bits needed to describe the selected predictor. H_T is the data part: expected bits needed to encode data with that predictor. Their sum $\text{MDL}_T(X) := S_T(X) + H_T(X)$ is the minimum two-part description length under the bound.

Random variable X	Time bound	$S_T(X)$	$H_T(X)$
U_n , uniform on $\{0, 1\}^n$	enough for uniform model	$O(1)$	$n + O(1)$
0101... or 1010..., probability $\frac{1}{2}$ each	$T(n) = \Theta(n)$	$O(1)$	$O(1)$

For U_n , $S_T = O(1)$ and $H_T \geq n$. For the alternating pair, both parts are $O(1)$.

Two consequences of the time bound

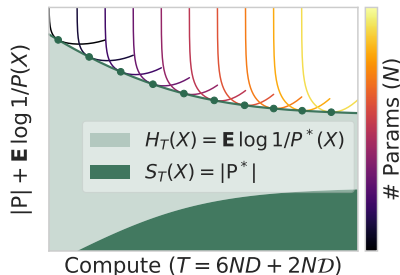
- **Pseudorandom generator.** A short seed is stretched into n bits. Shannon entropy sees only the seed; a polynomial-time observer sees almost n unpredictable bits and almost no structure.
- **One-way map.** For $Y = f(X)$, predicting Y from X is easy, while recovering X from Y is hard. A bounded observer assigns different information to the two orders.

For supervised learning we use the conditional version $S_T(Y | X)$: the input X is available, and only the labels Y are encoded.

The same data can look random or structured depending on what the observer can compute.

From programs to networks: the compute-optimal code

- Search over all programs is intractable, so fix an architecture and optimiser.
- Approximate dense-model accounting:
 $6ND + 2ND \leq T$, where N counts parameters and $D, \mathcal{D}$ count training and evaluation tokens.
- Each run gives a model code plus a validation-data code. Select the shortest admissible two-part code; a lower hull smooths the empirical curve.
- Report its model and data parts as estimates $\hat{S}_T$ and $\hat{H}_T$.

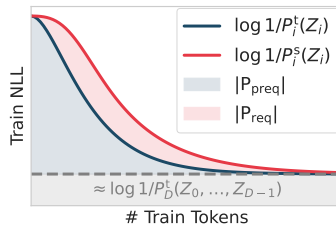


Schematic. Thin curves: training runs, coloured by N . Dots: the lower envelope. Dark green: S_T ; light green: H_T . Source: Fig. 2b of Finzi et al. (2026).

Estimation I: the prequential code

- P_i has seen $Z_0, \dots, Z_{i-1}$; P_0 is the initial model.
- Encode Z_i at ideal cost $-\log_2 P_i(Z_i)$, then update. The decoder repeats the update.
- The sequential data code sums these costs. Subtract the final-model data code to estimate the model part:

$$\hat{K}_{\text{preq}} = \sum_{i < M} [-\log P_i(Z_i) + \log P_M(Z_i)].$$



Dark curve: $\log 1/P_i(Z_i)$ of the network trained on real data (superscript t). Blue area: $|P_{\text{preq}}|$. Red curve and red area: the prequential code of the next slide.
Source: Fig. 2a of Finzi et al. (2026).

- The loss is measured before fitting each example, so it is a test-like loss.
- The area is a heuristic model-part estimate; subtraction alone gives no model-only code.

Estimation II: the requential code

- A teacher model trains on real data; a student model trains on teacher-distributed samples (Qiu et al., 2026).
- Both sides share the student and a random seed. Send the index of a student proposal selected to have the teacher's distribution.
- Relative entropy coding bounds the expected message cost by teacher–student disagreement:

$$\mathbb{E}[\ell_i \mid \mathcal{F}_i] \leq \text{KL}_i + 2 \log(1 + \text{KL}_i) + \kappa, \quad \kappa < 5.21,$$

where $\text{KL}_i = \text{KL}(P_i^t \| P_i^s)$ and $\mathcal{F}_i$ is the prior history. Summing gives the student-code bound; $\sum_i \text{KL}_i$ is its leading estimate.

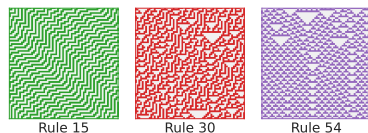
	Prequential	Requential
Model part	heuristic subtraction	explicit student reconstruction
Training	one network	teacher and student, 2–10× slower
Reported values	larger in the paper	smaller in the paper

Prequential is cheaper; requential has an explicit code.

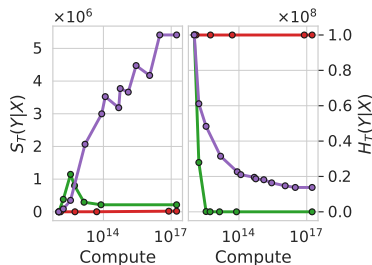
Experiment: cellular automata create structure

An *elementary cellular automaton* (ECA) updates binary cells in parallel from their three-cell neighbourhoods (Wolfram, 2002).

Setting. 64 periodic cells; $Y = F_r^{48}(X)$; transformer sweep; sequential code.



Rule	15	30	54
$S_T(Y X)$, bits	2×10^5	≈ 0	5.4×10^6
$H_T(Y X)$, bits	≈ 0	10^8	1.5×10^7

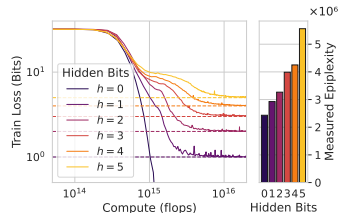
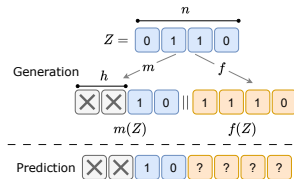


Left: rollouts, time downward. Right: S_T and H_T against compute in FLOPs. Table: our estimates from the plots at 10^{17} FLOPs. Source: Fig. 3 of Finzi et al. (2026).

Rule 15 is simple, rule 30 looks random, and rule 54 contains learnable structure.

Experiment: prediction requires more structure than generation

Setting. Hide h bits of a random $Z \in \{0,1\}^{32}$; predict four rule 30 updates of Z from the visible bits.



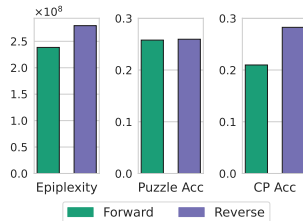
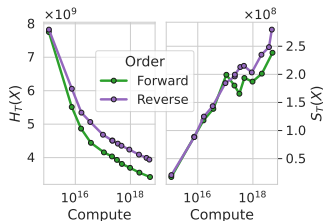
Left: the data. Right: training loss against compute in FLOPs (dashed: h bits) and S_T for each h . Source: Fig. 5 of Finzi et al. (2026).

- The generator runs forward; the predictor must infer the hidden bits.
- The loss approaches h bits only after rapidly growing compute, and S_T grows with h .

Predicting with partial observations requires inference that is absent from forward generation.

Experiment: the order of chess data

Setting. Lichess games in two orders: moves $\rightarrow$ final board, or board $\rightarrow$ moves. Transformer sweep with sequential coding.



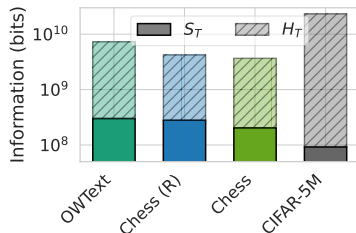
Left: H_T and S_T against compute in FLOPs. Right: S_T and accuracy after fine-tuning. Source: Figs. 4c and 7 of Finzi et al. (2026).

- At large compute, reverse order gives higher H_T and S_T .
- It transfers better to board evaluation: 0.28 vs 0.21 accuracy; puzzles are tied.

The reverse order forces a model of the board state. That structure transfers to a task on board states.

Application: structure in natural data

Setting. Equal token budgets and a shared compute bound. Compare OpenWebText, chess and CIFAR-5M with the sequential code.



Chess (R): reverse order. Source: Fig. 8a of Finzi et al. (2026). Table: our estimates from the figure.

Dataset	S_T , bits	MDL_T , bits	S_T/MDL_T
OpenWebText	3×10^8	7×10^9	4%
Chess, reverse	2.8×10^8	4×10^9	7%
Chess, forward	2×10^8	3.5×10^9	6%
CIFAR-5M	9×10^7	2.3×10^{10}	0.4%

- Under this protocol, text has the largest estimated model part; pixel images have the largest residual part.
- Scaling laws and adaptive data selection show the same ordering (Jiang et al., 2025).

This ranking is relative to the tokenisation, model family and compute budget.

Summary and limitations

Summary

- S_T counts selected model bits; H_T is the expected data-code length.
- A time bound makes computation, prediction order and learned structure observable.
- Estimate S_T with prequential area or teacher–student divergence at the compute-optimal model.

Limitations

- S_T is not a guarantee of downstream usefulness.
- The theorems are asymptotic and cryptographic; the proved lower bound is only $\Omega(\log n)$.
- Estimates depend on architecture, optimiser, T , dataset size and random seed.

Literature

M. Finzi et al. *From Entropy to Epilepsy*. arXiv:2601.03220v2, 2026.

S. Qiu et al. *Requential Coding*. arXiv:2607.11883, 2026.

M. Li, P. Vitányi. *Kolmogorov Complexity*. Springer, 2008.

S. Wolfram. *A New Kind of Science*. 2002.

Y. Jiang et al. Adaptive data optimization. *ICLR*, 2025.

Questions for the audience: answers

1. A dataset X consists of n uniformly random bits. Which quantity is close to n ?

Answer

$H_T(X) \approx n$: the bounded observer cannot predict the bits. A constant-size uniform model gives $S_T(X) = O(1)$.

2. In the prequential estimate, which part of the training-loss curve approximates the epiplexity?

Answer

The area between the loss before each update and the final-model loss, namely $\sum_i [\log 1/P_i(Z_i) - \log 1/P_M(Z_i)]$.

3. What does changing T change?

Answer

It changes the available observer. The total MDL_T cannot increase with T , while S_T itself may rise or fall.

Backup: basic properties

Proposition 1 (Sec. 3 of the paper)

Let c_1, c_2 be constants independent of X . For a random variable X on $\{0, 1\}^n$:

1. $S_T(X) \geq 0$ and $H_T(X) \geq 0$;
2. $H(X) \leq \text{MDL}_T(X) \leq n + c_1$;
3. $\text{MDL}_{T'}(X) \leq \text{MDL}_T(X)$ whenever $T'(n) \geq T(n)$;
4. $\text{MDL}_{T'}(f^{-1}(X)) \leq \text{MDL}_T(X) + |f| + c_2$ with $T'(n) = T(n) + \text{Time}(f)$, for a program f that computes a bijection f in fixed time.

Property 4 is the analogue of $K(f(x)) \leq K(x) + K(f) + c$. Under a time bound a short program for f^{-1} gives no short program for f .

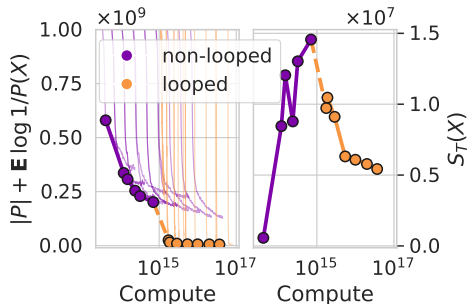
Theorem 1 (Thm. 10 of the paper)

If one-way functions secure against probabilistic polynomial-time algorithms with polynomial advice exist, then some random variables X_n on $\{0, 1\}^n$ have $S_{\text{Poly}}(X_n) = \Omega(\log n)$.

Backup: emergence, where S_T falls as T grows

Setting. ECA rule 54, 64 updates. A non-looped transformer predicts the final state directly. A looped transformer generates the intermediate states one after another: the exact simulation.

- Below 10^{15} FLOPs the non-looped model gives the shorter code. The epiplexity of this model rises to 1.45×10^7 bits: it learns emergent patterns of rule 54.
- Above that bound the looped model gives the shorter code. MDL_T drops to almost zero; S_T falls.
- The authors call this case uncommon. For natural data at moderate compute they expect S_T to grow with compute.



Two-part code and S_T against compute in FLOPs.
Source: Fig. 6 of Finzi et al. (2026).

With less compute the best model is a longer program: S_T need not grow with T .